

Interface Realtime Examples in C#

In this article, I am going to discuss **Interface Realtime Examples in C#** i.e. How to Use Interface in C# Application Development with Real-time Examples. Please read our previous article where we discussed [Interface Interview Questions and Answers in C#](#).

What is an Interface in C#?

The Interface in C# is a fully un-implemented class used for declaring a set of methods of an object. So, we can define an interface as a pure abstract class which allows us to define only abstract methods. The abstract method means a method without a body or implementation.

Its implementation must be provided by class or struct. The class or struct which implements the interface must provide the implementation of all the methods declared inside the interface.

Real-Time Example of Interface in C#:

If you are attending any interviews, then the interviewer might ask you this question to you what is the use of the interface in the real-time applications or he might be asked you where you have used the interface in your project?

Let us understand the answer with one real-time example. Suppose, you are going to develop an application for a Bank. And we know nowadays, we can open different types of bank accounts, for example, Saving Accounts, Current Accounts, Demat Accounts, etc. And we already know the features and services of all these different types of bank Accounts.

Now, the question is how we can implement this application. So, we are going to develop one Bank application where we will allow the user to create the following accounts.

1. **Saving Account**
2. **Current Account**

And in each account, we are going to provide the following services.

1. **DepositAmount**
2. **WithdrawAmount**
3. **CheckBalance**

So, how can implement the above means, we will create one interface, let's say IBankAccount by declaring above three services as follows:

```
namespace BankApplication
{
    interface IBankAccount
    {
        bool DepositAmount(decimal amount);
        bool WithdrawAmount(decimal amount);
        decimal CheckBalance();
    }
}
```

```
}  
}
```

Next, we need to create two different classes for Saving Account, Current Account, and Demat Account and we need to implement the above IBankAccount interface and provide the implementation for the above interface methods.

Saving Account:

Create a class file with the name SavingAccount.cs and then copy and paste the following code. Here, you can see that we are creating three private variables. The Balance variable is used to hold the account balance of the user, PerDayWithdrawLimit is used to restrict the user withdrawal i.e. we are setting 10000 per day and the TodayWithdrawal variable is used to hold the current day withdrawal amount. Our class is implementing the IBankAccount interface and providing the implementation for all three abstract methods that are defined inside the IBankAccount interface.

1. **DepositAmount:** This method takes the amount to be deposited and the logic is very straightforward. Whatever amount we are getting, we are just adding the amount with the Balance private variable.
2. **WithdrawAmount:** In this method, we have written three pieces of logic. First, we are checking whether the withdrawal amount is less than the available balance or not. Second, we are checking whether the withdrawal amount exceeds the per-day withdrawal limit or not. In these two conditions are satisfied, then we are withdrawing the amount and returning true to the user.
3. **CheckBalance:** This method implementation is very straightforward; whatever value we have in the Balance variable we are simply returning.

The complete class code is given below.

```
using System;  
namespace BankApplication  
{  
    public class SavingAccount : IBankAccount  
    {  
        private decimal Balance = 0;  
        private readonly decimal PerDayWithdrawLimit = 10000;  
        private decimal TodayWithdrawal = 0;  
        public bool DepositAmount(decimal Amount)  
        {  
            Balance = Balance + Amount;  
            Console.WriteLine($"You have Deposited: {Amount}");  
            Console.WriteLine($"Your Account Balance: {Balance}");  
            return true;  
        }  
        //Maximum Withdrawal Per Day: 10000  
        public bool WithdrawAmount(decimal Amount)  
        {  
            if (Balance < Amount)  
            {  
                Console.WriteLine("You have Insufficient balance!");  
                return false;  
            }  
            else if (TodayWithdrawal + Amount > PerDayWithdrawLimit)
```

```

        {
            Console.WriteLine("Withdrawal attempt failed!");
            return false;
        }
        else
        {
            Balance = Balance - Amount;
            TodayWithdrawal = TodayWithdrawal + Amount;
            Console.WriteLine($"You have Successfully Withdraw: {Amount}");
            Console.WriteLine($"Your Account Balance: {Balance}");
            return true;
        }
    }
    public decimal CheckBalance()
    {
        return Balance;
    }
}

```

Current Account:

Next, create a class file with the name CurrentAccount.cs and then copy and paste the following code. Here, you can see that we are creating one private variable i.e. Balance to hold the account balance of the user. Here, we are not creating PerDayWithdrawLimit and TodayWithdrawal variables as there is no limit on the amount to be withdrawn from a current account. Then our class CurrentAccount is implementing the IBankAccount interface and provides the implementation for all three abstract methods that are defined inside the IBankAccount interface.

1. **DepositAmount:** This method takes the amount to be deposited and the logic is the same as the SavingAccount. Whatever amount the user is sending, we are just adding the amount with the Balance private variable.
2. **WithdrawAmount:** In this method, first, we are checking whether the withdrawal amount is less than the available balance or not. If the withdrawal amount is less than the account balance, then we are withdrawing the amount and returning true to the user.
3. **CheckBalance:** This method implementation is the same as SavingAccount; whatever value we have in the Balance variable that we are simply returning.

The complete class code is given below.

```

using System;
namespace BankApplication
{
    public class CurrentAccount : IBankAccount
    {
        private decimal Balance = 0;

        public bool DepositAmount(decimal Amount)
        {
            Balance = Balance + Amount;
            Console.WriteLine($"You have Deposited: {Amount}");
        }
    }
}

```

```

        Console.WriteLine($"Your Account Balance: {Balance}");
        return true;
    }
    public bool WithdrawAmount(decimal Amount)
    {
        if (Balance < Amount)
        {
            Console.WriteLine("You have Insufficient balance!");
            return false;
        }
        else
        {
            Balance = Balance - Amount;
            Console.WriteLine($"You have Successfully Withdraw: {Amount}");
            Console.WriteLine($"Your Account Balance: {Balance}");
            return true;
        }
    }
    public decimal CheckBalance()
    {
        return Balance;
    }
}

```

Now, modify the Main method of the Program class as follows to test the functionality.

```

using System;
namespace BankApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Saving Account:");
            IBankAccount savingAccount = new SavingAccount();
            savingAccount.DepositAmount(2000);
            savingAccount.DepositAmount(1000);
            savingAccount.WithdrawAmount(1500);
            savingAccount.WithdrawAmount(5000);
            Console.WriteLine($"Saving Account Balanace:
{savingAccount.CheckBalance()}");
            Console.WriteLine("\nCurrent Account:");
            IBankAccount currentAccount = new CurrentAccount();
            currentAccount.DepositAmount(500);
            currentAccount.DepositAmount(1500);
            currentAccount.WithdrawAmount(2600);
            currentAccount.WithdrawAmount(1000);
            Console.WriteLine($"Current Account Balanace:
{currentAccount.CheckBalance()}");
        }
    }
}

```

```
        Console.ReadLine());  
    }  
}  
}
```

Output:

```
Saving Account:  
You have Deposited: 2000  
Your Account Balance: 2000  
You have Deposited: 1000  
Your Account Balance: 3000  
You have Successfully Withdraw: 1500  
Your Account Balance: 1500  
You have Insufficient balance!  
Saving Account Balanace: 1500  
  
Current Account:  
You have Deposited: 500  
Your Account Balance: 500  
You have Deposited: 1500  
Your Account Balance: 2000  
You have Insufficient balance!  
You have Successfully Withdraw: 1000  
Your Account Balance: 1000  
Current Account Balanace: 1000
```

In the next article, I am going to discuss [Multiple Inheritance in C#](#) with Examples. Here, in this article, I try to explain **Interface Realtime Examples in C#**. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Interface Realtime Examples in the C# article.